

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA0609	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Ranjitha. R	Reg. No.:	192421285
Section / Batch:	B.Tech Information Technology	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

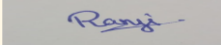
Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I Ranjitha. R (192421285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



AIM

To implement the Minimum Cost Climbing Stairs problem using Dynamic Programming and determine the minimum cost required to reach the top when a person can climb either one step or two steps at a time.

ALGORITHM

1. Start the program.
2. Read the number of steps n .
3. Read the cost of each step into the array $\text{cost}[]$.
4. Create a DP array $\text{dp}[]$ of size $n + 1$.
5. Initialize $\text{dp}[0] = 0$ and $\text{dp}[1] = 0$, since the person can start from either the first or second step.
6. For each position from 2 to n , calculate:
7. $\text{dp}[i] = \min(\text{dp}[i-1] + \text{cost}[i-1], \text{dp}[i-2] + \text{cost}[i-2])$
8. Continue until the top is reached.
9. The value $\text{dp}[n]$ represents the minimum cost required to reach the top.
10. Determine the path that gives the minimum cost.
11. Display the minimum cost and optimal path.
12. Stop the program.

PROCEDURE

1. Open the Python environment or compiler.
2. Create a new program for the Minimum Cost Climbing Stairs problem.
3. Enter the number of steps.
4. Enter the cost associated with each step.
5. Store the values in the $\text{cost}[]$ array.
6. Initialize the DP array with $\text{dp}[0] = 0$ and $\text{dp}[1] = 0$.
7. For each step, calculate the minimum cost by comparing the cost of reaching it from the previous step and from two steps before.
8. Store the smaller value in the corresponding $\text{dp}[]$ position.
9. Repeat the calculation until the final step is processed.
10. Compare the possible routes and identify the minimum-cost route.
11. Display the DP values, optimal path, and minimum cost.
12. Verify the result with the given input and stop the program.

MANUAL STEP-BY-STEP PROCESS

Given Input

Number of steps = 3

Cost = [43, 32, 12]

This means:

- Step 1 costs 43
- Step 2 costs 32
- Step 3 costs 12
- The person can climb 1 step or 2 steps at a time.
- The person can start from Step 1 or Step 2.

STAGE 1 • INITIAL SETUP

STEP 1: INPUT VALUES

We begin by inspecting the input staircase parameters and understanding the initial condition rules.

1.1 CONFIGURED INPUTS

Number of Steps (n):

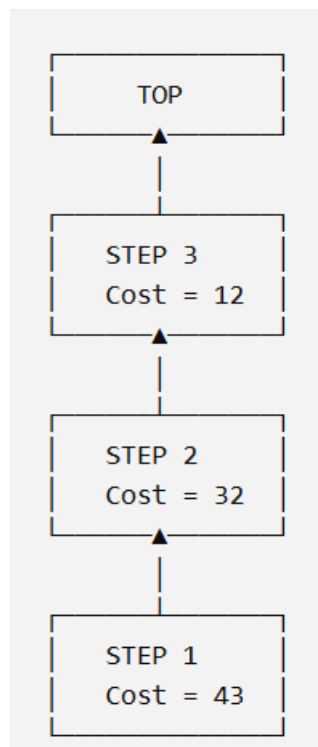
Cost Array:

1.2 STEP COST BREAKDOWN

1	Step 1	₹43
2	Step 2	₹32
3	Step 3	₹12

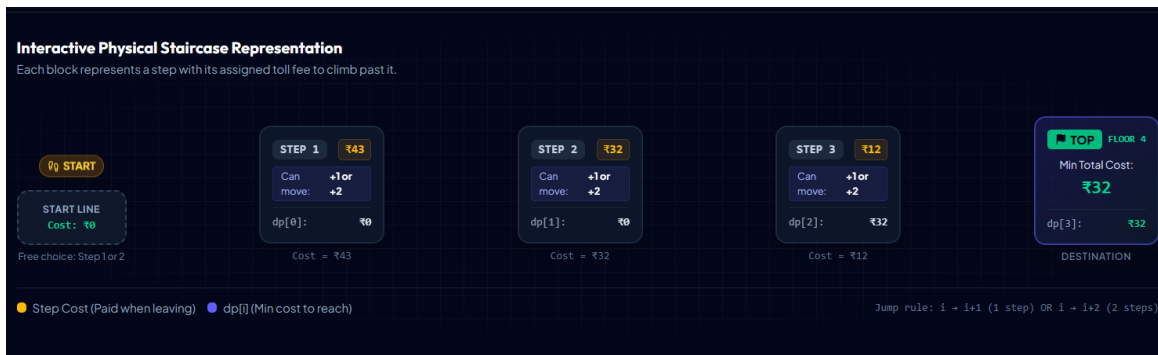
Step 1: Represent the Staircase

The staircase can be represented as:



The person can move:

- Step 1 → Step 2 (1 step)
- Step 1 → Step 3 (2 steps)
- Step 2 → Step 3 (1 step)
- Step 2 → TOP (2 steps)



Step 2: Initialize the DP Array

We create a DP array to store the minimum cost required to reach each position.

Initially:

$$dp[0] = 0$$

$$dp[1] = 0$$

Why is $dp[1] = 0$?

The problem allows the person to start from either Step 1 or Step 2. Therefore, no cost is added before choosing the starting step.

Initial DP table:

Position	Cost	DP Value
Start	—	0
Step 1	43	0
Step 2	32	—
Step 3	12	—



Step 3: Calculate the Cost to Reach Step 2

There are two choices for reaching the position corresponding to Step 2.

Choice 1: Start from Step 1

The cost is:

$$\begin{aligned} & dp[1] + \text{cost}[1] \\ &= 0 + 32 \\ &= 32 \end{aligned}$$

Choice 2: Start directly from Step 2

The cost is:

$$\begin{aligned} & dp[0] + \text{cost}[0] \\ &= 0 + 43 \\ &= 43 \end{aligned}$$

Now compare:

32 vs 43

The smaller value is:

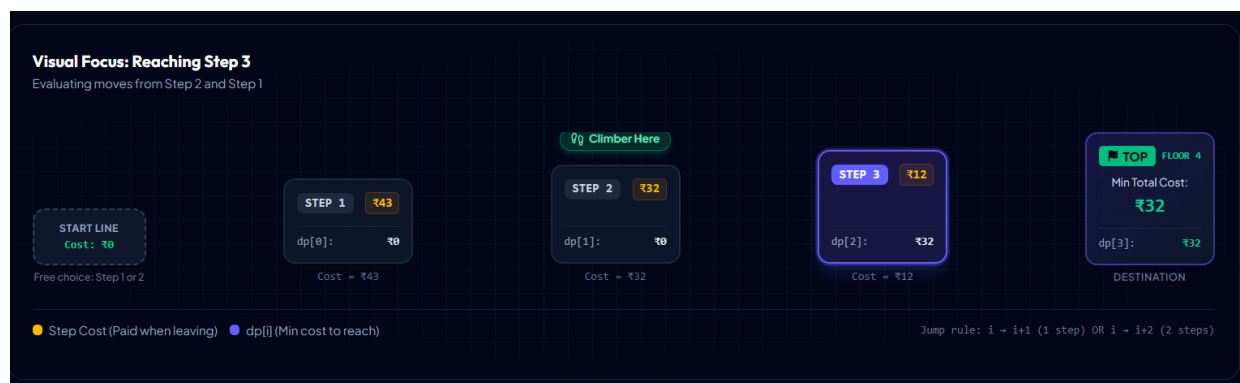
32

Therefore:

$$dp[2] = 32$$

Updated table:

Position	Cost	DP Value
Start	—	0
Step 1	43	0
Step 2	32	32
Step 3	12	—



Step 4: Calculate the Cost to Reach Step 3 / Top

Now we calculate dp[3].

There are two possible ways to reach the top.

Choice 1: Move from Step 3

The person reaches Step 3 first and then reaches the top.

Cost:

$$\begin{aligned}
 & dp[2] + \text{cost}[2] \\
 &= 32 + 12 \\
 &= 44
 \end{aligned}$$

So this option costs 44.

Choice 2: Move from Step 2 Directly to the Top

The person starts at Step 2 and jumps directly to the top.

Cost:

$$\begin{aligned}
 & dp[1] + \text{cost}[1] \\
 &= 0 + 32 \\
 &= 32
 \end{aligned}$$

So this option costs 32.

Compare Both Choices

Option 1 = 44

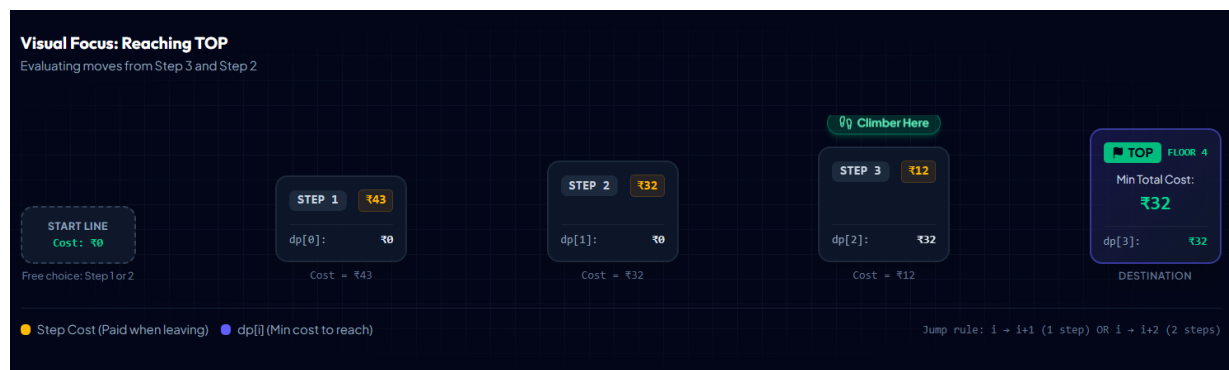
Option 2 = 32

Choose the smaller value:

$$\begin{aligned}
 dp[3] &= \min(44, 32) \\
 &= 32
 \end{aligned}$$

Therefore:

Minimum cost to reach the top = 32



Step 5: Complete DP Table

After all calculations:

Position	Step Cost	Minimum Cost
Start	—	0
Step 1	43	0
Step 2	32	32
Step 3	12	32

Position	Step Cost	Minimum Cost
Top	—	32

The final DP value is:

$$dp[3] = 32$$

FINAL DYNAMIC PROGRAMMING STATE TABLE				All States Solved
POSITION	STEP COST (cost[i])	MINIMUM COST TO REACH (dp[i])	FORMULA NOTE	
Start (Floor 0)	-	₹0	Initial ground baseline	
Step 1 (index 0)	₹43	₹0	dp[0] = 0 (Base case: start here free)	
Step 2 (index 1)	₹32	₹0	dp[1] = 0 (Base case: start here free)	
Step 3 (index 2)	₹12	₹32	min(dp[1]+cost[1], dp[0]+cost[0])	
TOP (Floor 4)	-	₹32	Final destination minimum cost: ₹32	

Step 6: Check All Possible Routes

To understand the result clearly, we can manually compare all possible routes.

Route 1

START → STEP 1 → STEP 2 → TOP

Cost:

43 + 32
= 75

Route 1 = 75

Route 2

START → STEP 1 → STEP 3 → TOP

Cost:

43 + 12
= 55

Route 2 = 55

Route 3

START → STEP 2 → TOP

Cost:

32

Route 3 = 32

Step 7: Compare All Routes

Route	Path	Total Cost
Route 1	Step 1 → Step 2 → Top	75
Route 2	Step 1 → Step 3 → Top	55
Route 3	Step 2 → Top	32

Therefore:

$$\begin{aligned} \text{Minimum Cost} &= \min(75, 55, 32) \\ &= 32 \end{aligned}$$

ROUTE 1

OPTIMAL PATH

Trajectory:

START → STEP 2 → TOP

Tolls paid: ₹32

Calculation: 32

Total Route Cost: ₹32

ROUTE 2

Sub-optimal

Trajectory:

START → STEP 2 → STEP 3 → TOP

Tolls paid: ₹32 + ₹12

Calculation: 32 + 12

Total Route Cost: ₹44

ROUTE 3

Sub-optimal

Trajectory:

START → STEP 2 → STEP 3 → TOP

Tolls paid: ₹32 + ₹12

Calculation: 32 + 12

Total Route Cost: ₹44

ROUTE 4

Sub-optimal

Trajectory:

START → STEP 1 → STEP 3 → TOP

Tolls paid: ₹43 + ₹12

Calculation: 43 + 12

Total Route Cost: ₹55

ROUTE 5

Sub-optimal

Trajectory:

START → STEP 1 → STEP 3 → TOP

Tolls paid: ₹43 + ₹12

Calculation: 43 + 12

Total Route Cost: ₹55

ROUTE 6

Sub-optimal

Trajectory:

START → STEP 1 → STEP 2 → TOP

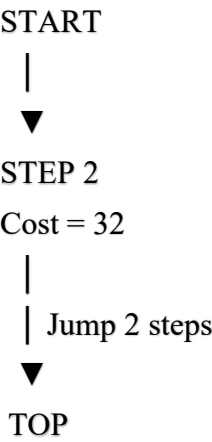
Tolls paid: ₹43 + ₹32

Calculation: 43 + 32

Total Route Cost: ₹75

Step 8: Identify the Optimal Path

The cheapest route is:



Therefore:

$$\text{Optimal Path} = \text{START} \rightarrow \text{STEP 2} \rightarrow \text{TOP}$$



Step 9: Apply the Dynamic Programming Formula

The general formula used is:

$$dp[i] = \min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2])$$

For this input:

For $dp[2]$

$$\begin{aligned} dp[2] &= \min(0 + 32, 0 + 43) \\ &= \min(32, 43) \\ &= 32 \end{aligned}$$

For $dp[3]$

$$\begin{aligned} dp[3] &= \min(32 + 12, 0 + 32) \\ &= \min(44, 32) \\ &= 32 \end{aligned}$$

Thus:

$$dp[3] = 32$$

Step 10: Final Conclusion

The Dynamic Programming method compares the cost of reaching each position through the available previous positions and stores the minimum value.

For the given input:

Number of steps = 3

Costs = [43, 32, 12]

the three possible route costs are:

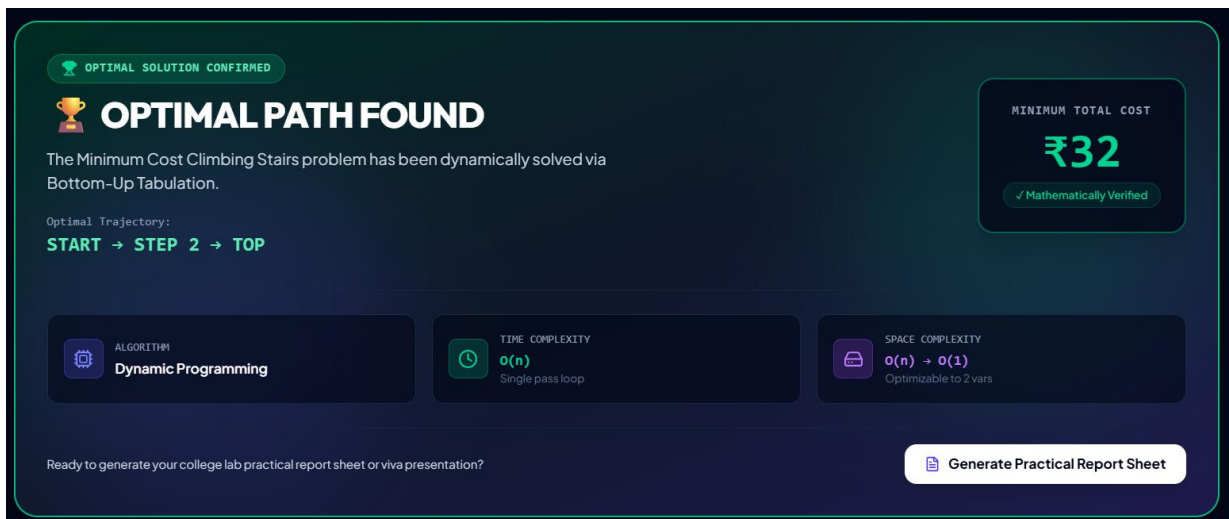
75, 55, 32

The smallest value is 32.

Final Result

Optimal Path: START → STEP 2 → TOP

Minimum Cost: 32



CONCLUSION

The Minimum Cost Climbing Stairs problem was successfully implemented using the Dynamic Programming technique. The method calculates the minimum cost for reaching each step by comparing the costs of coming from the previous one or two steps and storing the minimum value.

For the given input:

Number of Steps = 3

Cost = [43, 32, 12]

the possible routes were evaluated, and the minimum-cost path was found to be:

START → STEP 2 → TOP

with a minimum cost of 32.

The solution demonstrates how Dynamic Programming avoids unnecessary repeated calculations and efficiently finds the optimal solution. The algorithm has a time complexity of $O(n)$ and space complexity of $O(n)$.

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results

Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____